Socket Programming Fundamentals in C++

Includes: Key Includes: <sys/socket.h>, <netinet/in.h>, <arpa/inet.h>, <unistd.h> | **Note:** Windows uses Winsock2.h; Linux/Mac use socket.h

Basic Concepts

Socket Types:

- **SOCK_STREAM:** TCP (reliable, ordered)
- **SOCK_DGRAM:** UDP (fast, unreliable)

Address Family:

- **AF_INET:** IPv4
- **AF_INET6:** IPv6

Key Functions:

- **socket():** Create socket
- **bind():** Assign address
- **listen():** Listen for connections
- **accept():** Accept client
- **connect():** Connect to server
- **send()/recv():** Data transfer
- **close():** Close socket

Server Side (TCP)

Setup Steps:

1. Create socket
2. Bind address
3. Listen for connections
4. Accept connections
5. Send/Receive data

```
#include <iostream>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>

int main() {
  int server_fd = socket(AF_INET,
                         SOCK_STREAM, 0);

  sockaddr_in address;
  address.sin_family = AF_INET;
  address.sin_addr.s_addr = INADDR_ANY;
  address.sin_port = htons(8080);

  bind(server_fd, (sockaddr*)&address,
       sizeof(address));

  listen(server_fd, 5);

  int client_fd = accept(server_fd,
                         nullptr, nullptr);

  char buffer[1024] = {0};
  recv(client_fd, buffer,
       sizeof(buffer), 0);

  send(client_fd, "Hello Client", 12, 0);

  close(client_fd);
  close(server_fd);
  return 0;
}
```

htons(): Host to network byte order (endianness)

Client Side (TCP)

Setup Steps:

1. Create socket
2. Connect to server
3. Send/Receive data
4. Close socket

```
#include <iostream>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>

int main() {
  int sock = socket(AF_INET,
                    SOCK_STREAM, 0);

  sockaddr_in serv_addr;
  serv_addr.sin_family = AF_INET;
  serv_addr.sin_port = htons(8080);

  inet_pton(AF_INET, "127.0.0.1",
            &serv_addr.sin_addr);

  connect(sock, (sockaddr*)&serv_addr,
          sizeof(serv_addr));

  send(sock, "Hello Server", 12, 0);

  char buffer[1024] = {0};
  recv(sock, buffer, sizeof(buffer), 0);

  close(sock);
  return 0;
}
```

inet_pton(): Convert IP string to binary format

Function Reference

Function	Purpose
socket()	Create socket descriptor
bind()	Bind to address/port
listen()	Listen for incoming
accept()	Accept connection
connect()	Connect to server
send()	Send data
recv()	Receive data
close()	Close socket

Error Handling

```
if (socket_fd < 0) {
  perror("Socket failed");
  return -1;
}

if (connect(sock, ...) < 0) {
  std::cerr << "Connection failed"
            << std::endl;
  close(sock);
}
```

perror(): Print system error | **Return -1:** Indicates failure

Quick Reference

- Port 0-1023: System reserved
- Port 1024-49151: User ports
- Port 49152-65535: Dynamic ports
- **htons():** Convert port to network byte
- **shutdown():** Close one direction
- **SO_REUSEADDR:** Reuse port immediately